



Day 3 — SCM Integration, Builds & Quality

Jenkins CI/CD Training · Source Control · Automated Builds · Testing ·
Artifacts · Quality Gates

Learning Objectives

- **SCM Integration**

Integrate Jenkins with Git/GitHub, checkout source code, and understand Webhooks vs Polling

- **Automated Triggers**

Trigger builds automatically and understand Multibranch Pipelines

- **Builds & Testing**

Execute Maven builds, run unit tests, and publish test results

- **Artifacts**

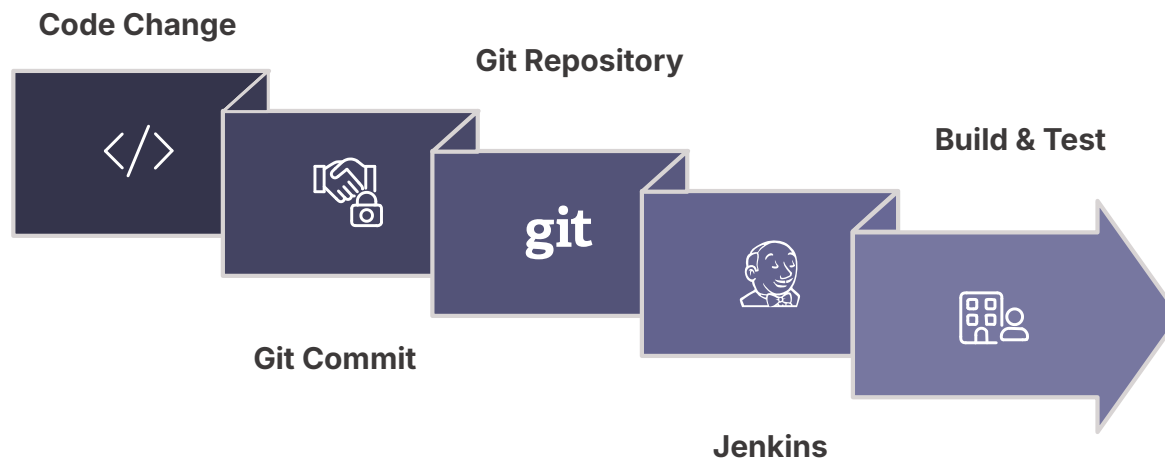
Archive build artifacts and understand build versioning

- **Quality Gates**

Explain quality gates and understand where SonarQube fits in CI/CD

 **Hands-on:** Git → Jenkins → Maven → Test → Artifact

Why Integrate Jenkins with SCM?



SCM Integration Enables

→ Automated Checkout

Source code retrieved automatically on every trigger

→ Pipeline-as-Code

Jenkinsfile lives alongside application code

→ Traceability & Repeatability

Every build tied to a specific commit

📌 **Key Idea:** The repository becomes the starting point of the CI workflow.

Git/GitHub Integration with Jenkins



Typical Integration Flow



Jenkins Needs

- Repository URL and Git on the execution node
- Repository access credentials (private repos)
- Jenkinsfile for Pipeline-as-Code

SCM Checkout

What is Checkout?

Checkout retrieves application source code from the configured repository into the Jenkins workspace, where build tools can then operate on it.



Declarative Pipeline Example

```
stage('Checkout') {  
  steps {  
    git branch: 'main',  
        url: 'https://github.com/  
example/sample-app.git'  
  }  
}
```

From Manual to Automated Builds

✗ Manual Build

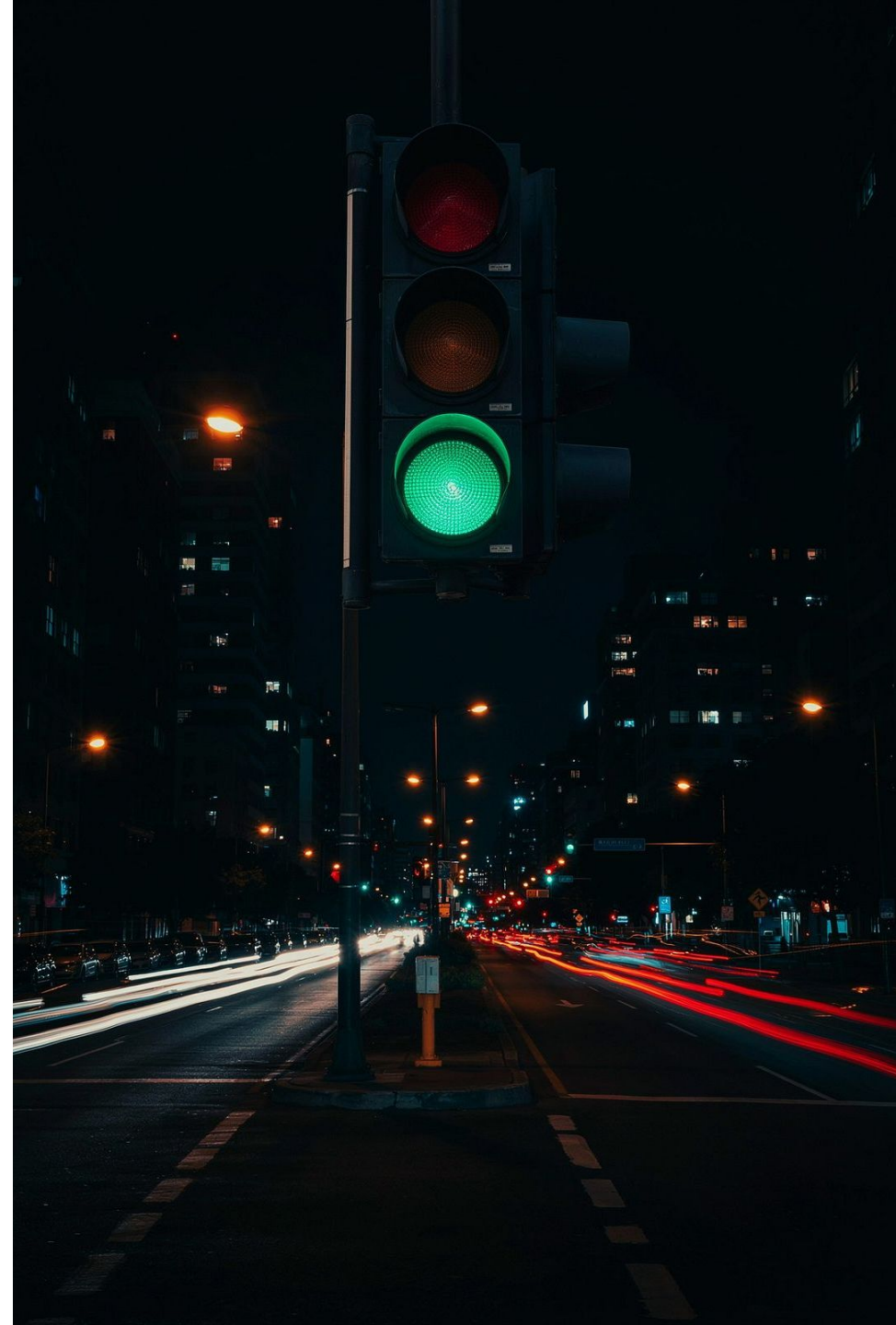
Push Code → Open Jenkins → Click "Build Now"

This works, but it is **not** continuous integration.

✓ Automated Build

Push Code → Repository Event → Jenkins Triggered → Build → Test

Every relevant code change should receive automated feedback.



Polling vs Webhooks

SCM Polling

Jenkins periodically checks the repository for changes.

```
triggers {  
  pollSCM('H/5 * * * *')  
}
```

Jenkins → "Any changes?" → Git Repository

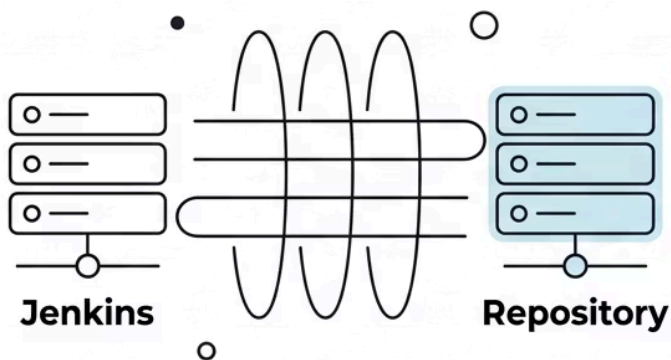
Webhook

The repository notifies Jenkins when an event occurs.

Developer → git **push**
→ GitHub → Webhook → Jenkins

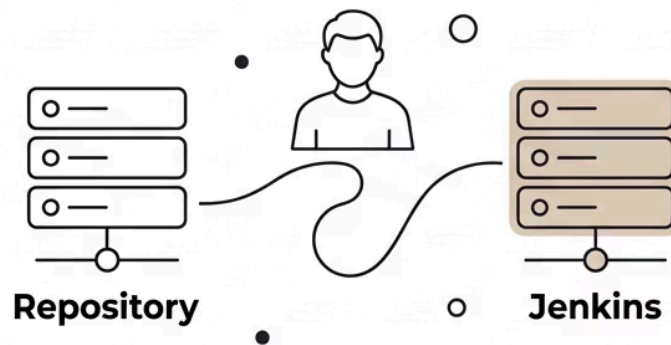
❏ Webhooks are commonly preferred where the repository and Jenkins environment support them.

Webhook vs Polling — Comparison



POLLING (Jenkins checks)

- Periodic checks (accent color fill #1B1B27)
- Can introduce delay
- Repeated repository checks



WEBHOOK (Repository notifies)

- Event-driven (accent color fill #1B1B27)
- Faster response
- Triggered on configured events

In a typical CI approach: **git push** → **Webhook** → **Jenkins** → **Pipeline**

Multibranch Pipeline Overview

The Challenge

Applications contain multiple branches. Do we manually create a Jenkins job for every branch?

Repository

- main
- develop
- feature/login
- feature/payment

The Solution

Jenkins discovers branches containing Jenkinsfiles and creates branch-specific Pipeline executions automatically.



 Day 3 scope: Conceptual overview only.



SECTION 4 — APPLICATION BUILDS

Automating Application Builds

After checkout, Jenkins invokes the application's build tool. For Java + Maven:

1

Checkout

Source retrieved from Git

2

Compile

Source code compiled

3

Test

Unit tests executed

4

Package

`mvn clean package`

Maven in a Jenkins Pipeline

Pipeline Stage

```
stage('Build') {  
  steps {  
    sh 'mvn clean package'  
    // Windows:  
    // bat 'mvn clean package'  
  }  
}
```

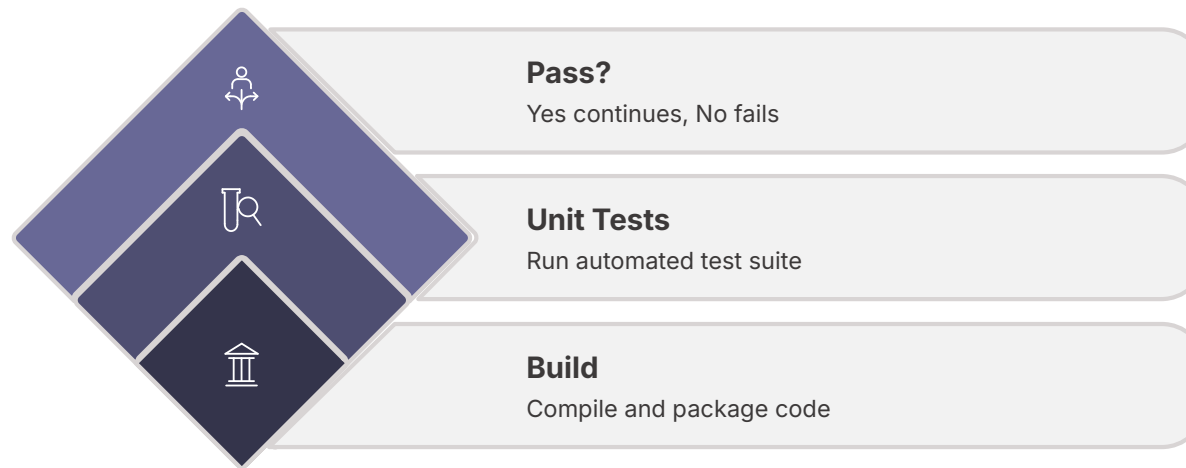
Successful Output

[INFO] BUILD SUCCESS

target/
└── sample-app-1.0.jar

✅ The `.jar` file produced is a **build artifact** — the deployable output of the pipeline.

Automated Tests in the Pipeline



Why Run Tests in CI?

Early Detection

Catch defects before they progress

Rapid Feedback

Developers notified immediately

Consistency

Same test execution every time

```
mvn test
```

Publishing Test Results

Pipeline Example

```
post {  
    always {  
        junit 'target/surefire-reports/*.xml'  
    }  
}
```

Running tests is only part of the process — Jenkins can also **publish and display** test reports directly from the build.

Jenkins Shows

Total Tests

Passed

Failed

History

Understanding Build Artifacts

An artifact is the deployable output produced from a validated build.



Java

`.jar` / `.war` files



.NET

Binaries and packages



Frontend

Distribution bundle



Archiving Artifacts & Build Versioning

Archiving Example

```
post {  
  success {  
    archiveArtifacts  
      artifacts: 'target/*.jar',  
      fingerprint: true  
  }  
}
```

Build History

Build #101 → sample-app-101.jar
Build #102 → sample-app-102.jar

 Day 4 extends this into **JFrog Artifactory**.

Build Versioning

Use meaningful version names to identify which build produced which artifact.

```
// Instead of:  
sample-app.jar
```

```
// Use:  
sample-app-1.0.105.jar
```

```
echo "Build ${env.BUILD_NUMBER}"
```

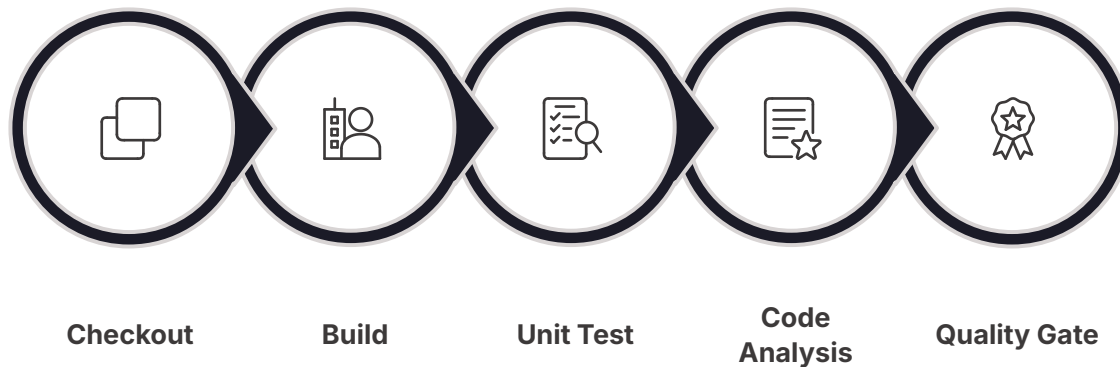
Preserve Outputs

Traceability

Auditability

Quality Gates & SonarQube

Passing unit tests does not mean code meets all quality expectations. SonarQube analyzes source code and evaluates configured quality conditions.



Quality Checks May Include

Bugs

Vulnerabilities

Code Smells

Duplicated Code

Test Coverage

Maintainability

i Day 3 scope: Brief quality-gate concept and SonarQube demo only — not deep administration.

Build a Basic CI Pipeline

Target Pipeline



Lab Steps

01

Use a Maven App

`jenkins-maven-demo/` with `src/`, `pom.xml`, `Jenkinsfile`

02

Configure SCM Checkout

Connect Jenkins to the Git repository

03

Build, Test & Package

`mvn clean compile` → `mvn test` → `mvn package`

04

Publish Results & Archive

Publish JUnit XML results and archive `target/*.jar`

05

Configure Automated Trigger

Code change initiates the build automatically

Test the CI Flow & Introduce a Failure

✓ Full CI Flow

Modify Code → git commit →
git push → Jenkins Triggered
→ Build → Tests → Artifact

✗ Introduce a Failure

Deliberately break a unit test.
Observe: Build → Test ✗ →
Pipeline Failure. Review
Console Output and published
test results.

⚙️ Optional Demo

Instructor shows SonarQube placement: Build → Test →
SonarQube Analysis → Quality Gate → Package

